WAFFLESTUDIO, FALL 2026

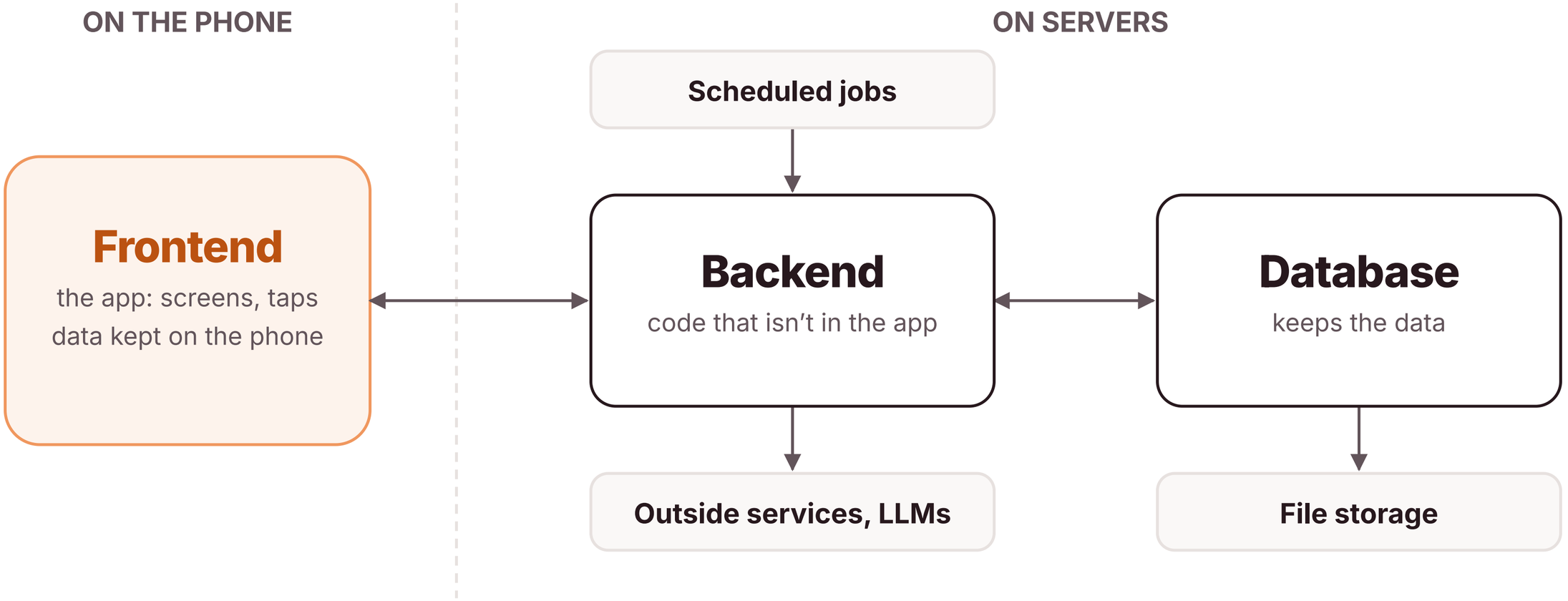
Product Engineering

Build end to end

Session 03, 2026/10/31
Chaehyun Park

LAST TIME

The parts of an app



LAST TIME

Four common shapes

A On the phone only

App + storage on the phone

B App + backend

App → functions → tables

C App + a function that holds a key

App → action → LLM, API

D Data collected on a schedule

cron → source → tables → App

Today: from the design to working code

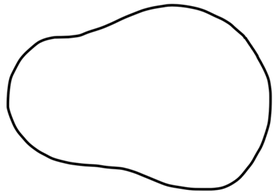
Divide the work → Pick the first slice → Build it with an agent → Test it

01

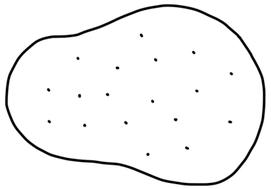
Divide the work

From one core flow to pieces you can finish

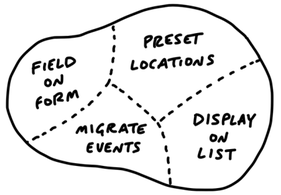
From a pile of tasks to scopes



The project



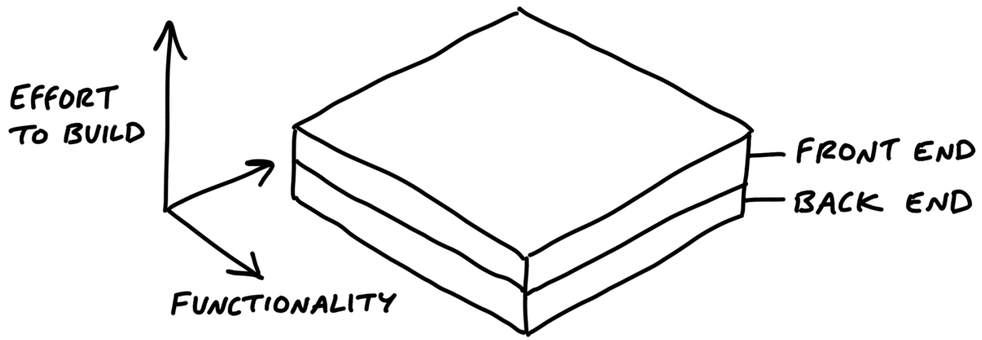
Tasks you find while working



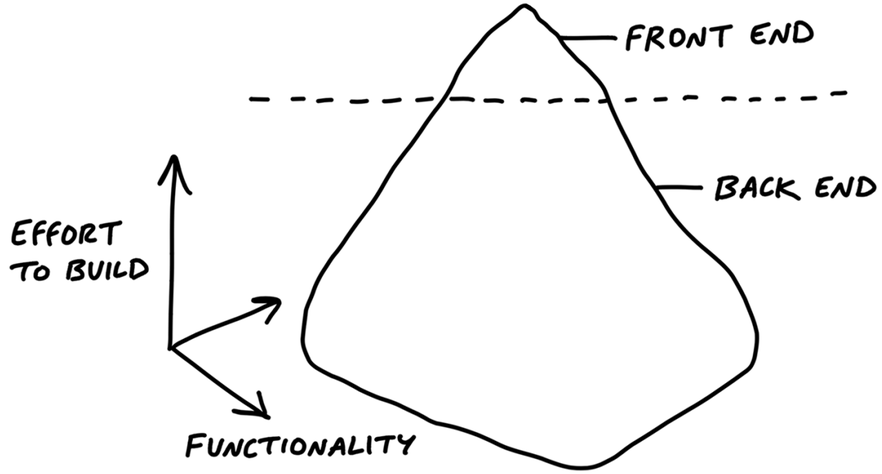
Scopes: pieces that finish on their own

A scope is a piece you can build and check front to back in a few days.

Two shapes of scope



Layer cake. A little backend under each screen. One scope per screen.



Iceberg. Most of the work hides under one screen, like an LLM call or a crawler. Give the hidden part its own scope.

YOUR FIRST STEP

Write the scopes down

docs/03-scopes.md

```
## Films nearby
- [ ] read today's films from the backend
- [ ] list them on the home screen
## Film page
- [ ] showtimes, nearest cinema first
- [ ] ~ poster opens full screen
## Saved
- [ ] save a film, see it in 보관함
```

Name scopes after what they do, not “frontend” or “bugs”. ~ marks a nice-to-have.

PRODUCTION: 무비바라

A written plan for each larger piece

```
docs/plans/
cicd-pipeline.md
movie-resolver.md
programme-ingestion.md
one-event-one-record.md
... 26 plans
```

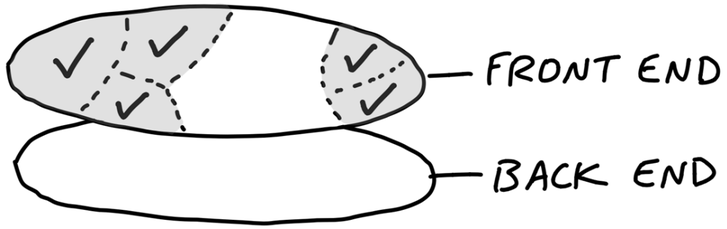
The agent writes the plan, I review it, and the code follows the plan.

02

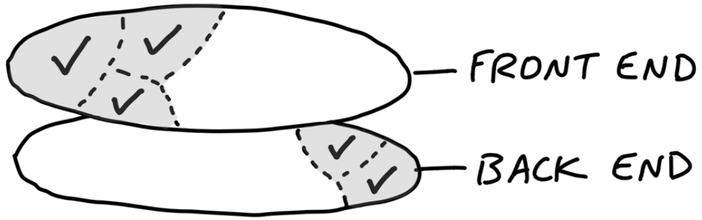
Pick the first slice

One thing that works for real, front to back

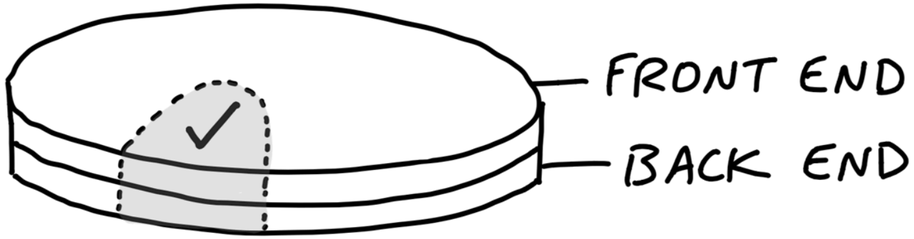
Build in slices, not layers



NOTHING WORKS



NOTHING WORKS



SOMETHING WORKS!

The first slice is core, small, new

Core

the rest
depends on it

Small

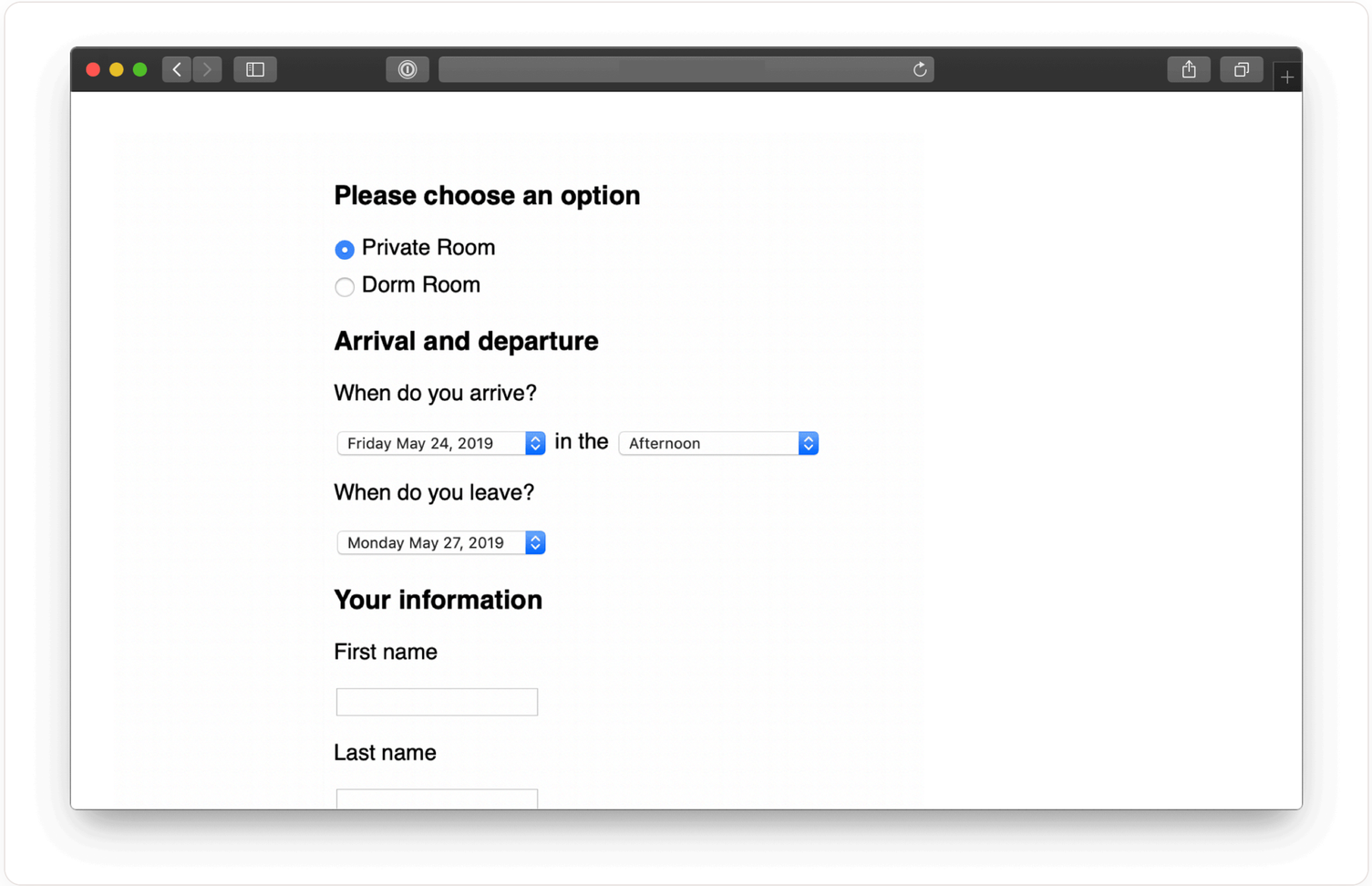
done in
a day or two

New to you

the part you're
least sure of

Start in the middle. Stub everything else: fake data, a plain button, no styling.

A first version can be unstyled



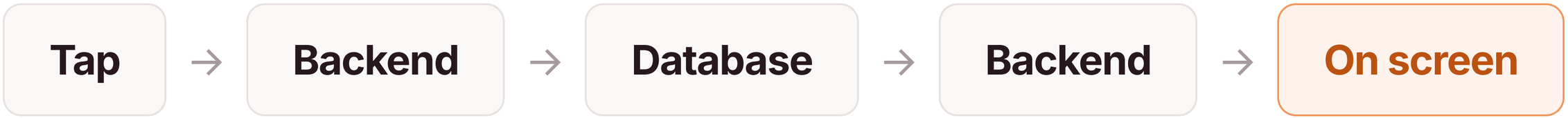
Plain fields wired to the real backend. Styling comes later.

Ryan Singer — Shape Up, ch.11 Get One Piece Done
basecamp.com/shapeup/3.2-chapter-11

YOUR FIRST STEP

Your first slice, in one sentence

```
When I ___, it is saved to ___,
and it shows on ___.
```



PRODUCTION: 무비바라 APP

Data first, then screens

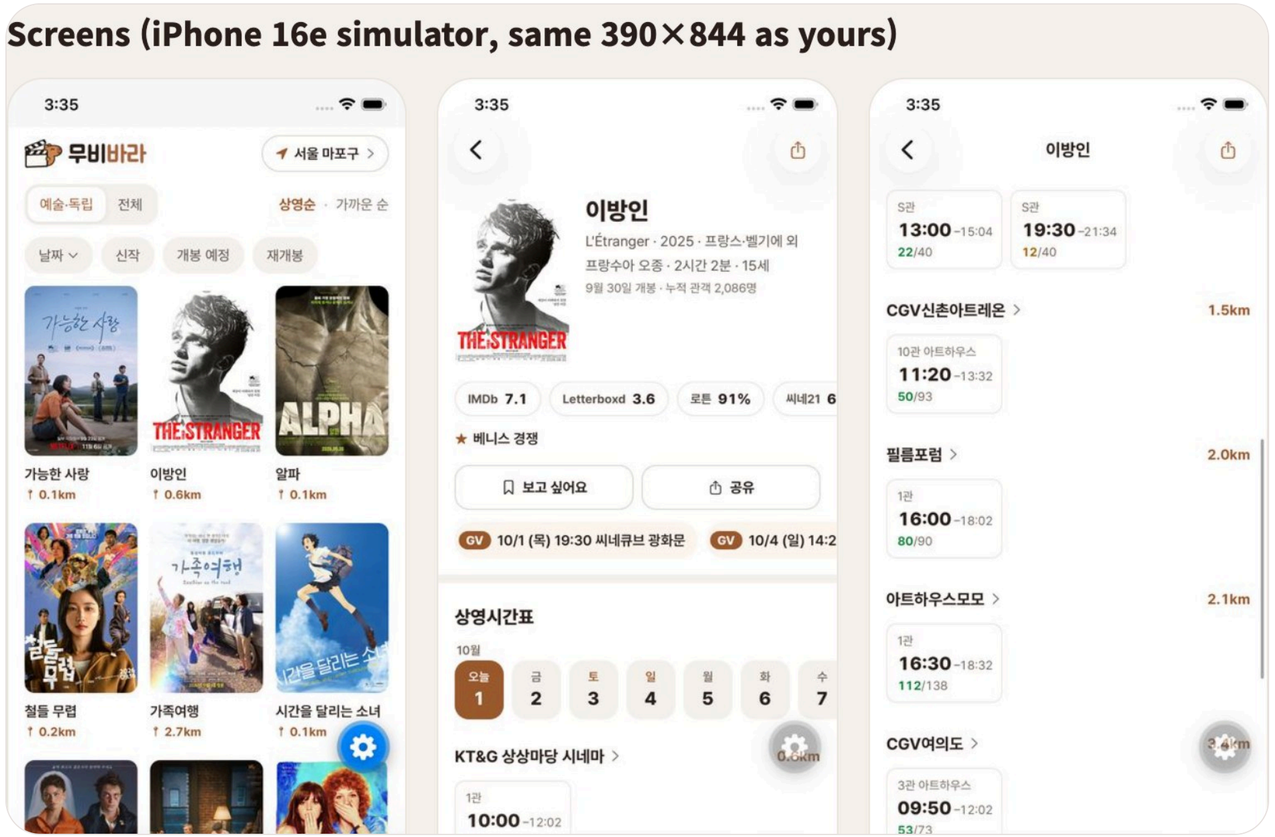
Read the data the website already publishes

↓

Put the main screens on top of it

↓

Loading and error states



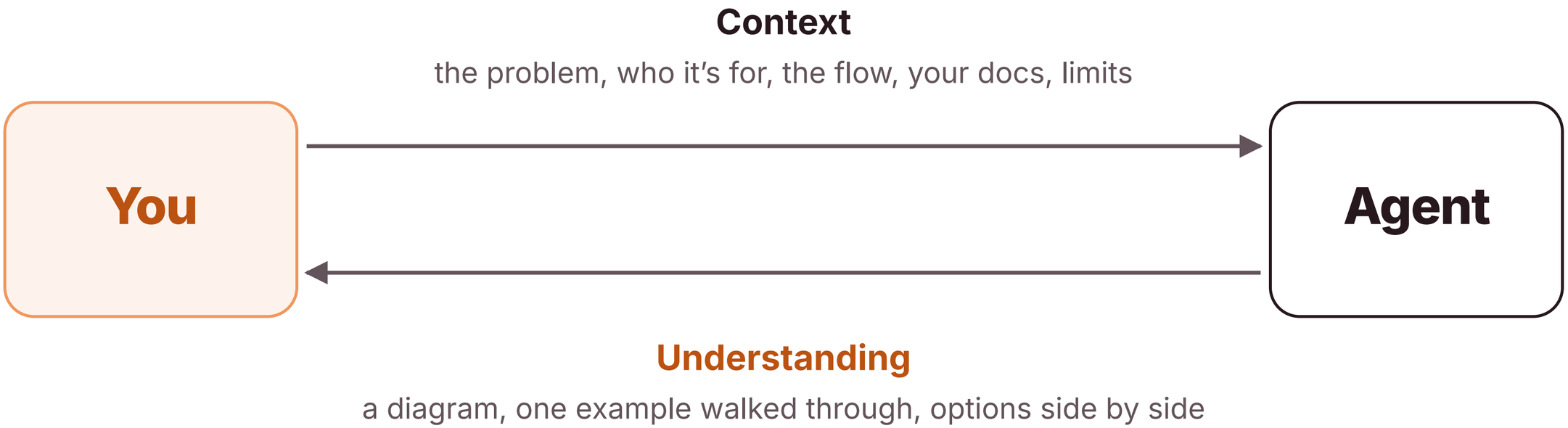
The first night, in order. Right: the agent's report the next morning.

03

Build it with an agent

Stay in sync, in both directions

Stay in sync with the agent



Do both before any code is written.

YOU → AGENT · 무비바라 ALERTS

Give it the full picture

MY PROMPT, EDITED

I want to add notifications. A user might want an alert for a film that isn’t showing anywhere yet, or for a film whose bookings haven’t opened at their cinemas on a certain day.

Go through the scenarios and what users would actually want. Let’s agree on those first, then work out how to build it. We need to stay on the free tier.

who it’s for | the scenarios | limits | the order: agree, then build

WHAT CAME BACK FIRST

Who wants what

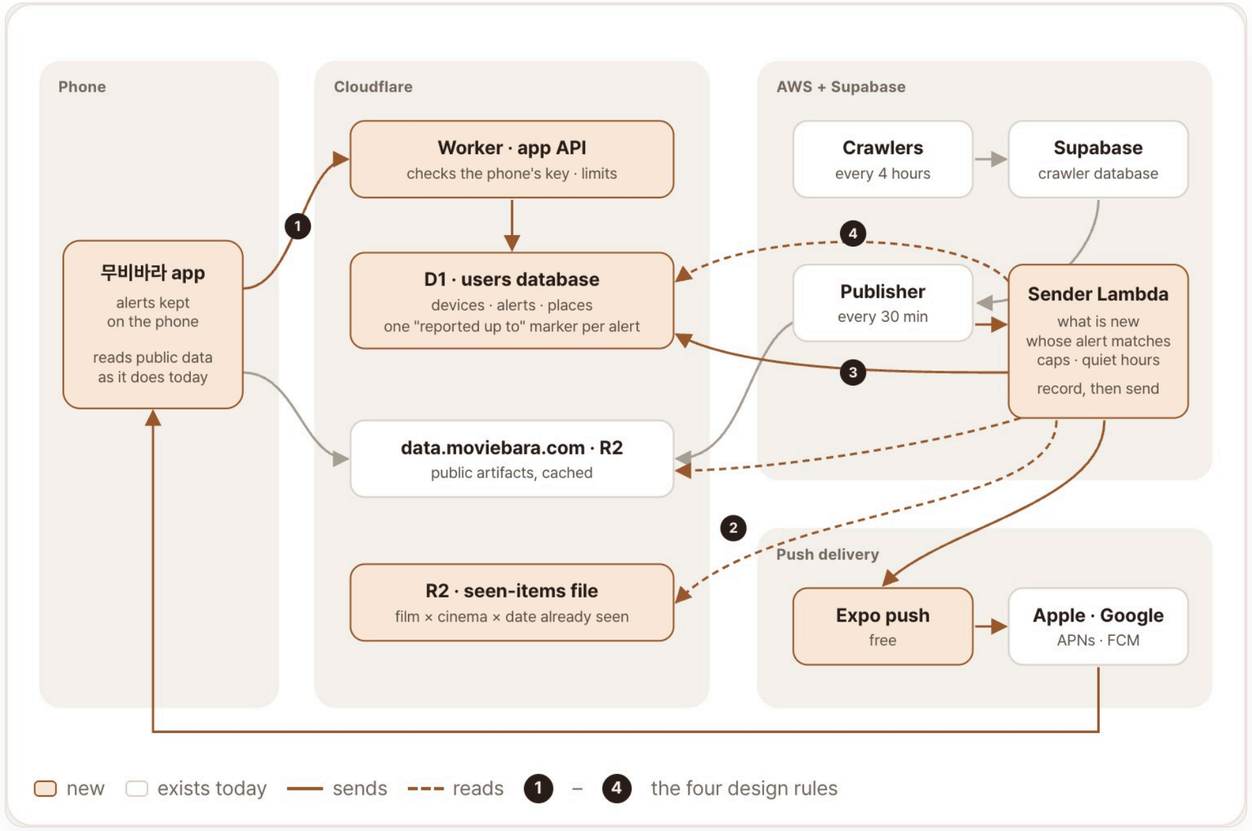
Who	Wants to hear when
Indie film fans	the film they saved starts playing near them
Cinephiles	a rare screening or restoration is scheduled
IMAX fans	new IMAX dates open for booking
Fans of a director	a talk with the director is announced

Simplified from the agent’s table. A wrong assumption is easy to fix at this stage.

AGENT → YOU · 무비바라 ALERTS

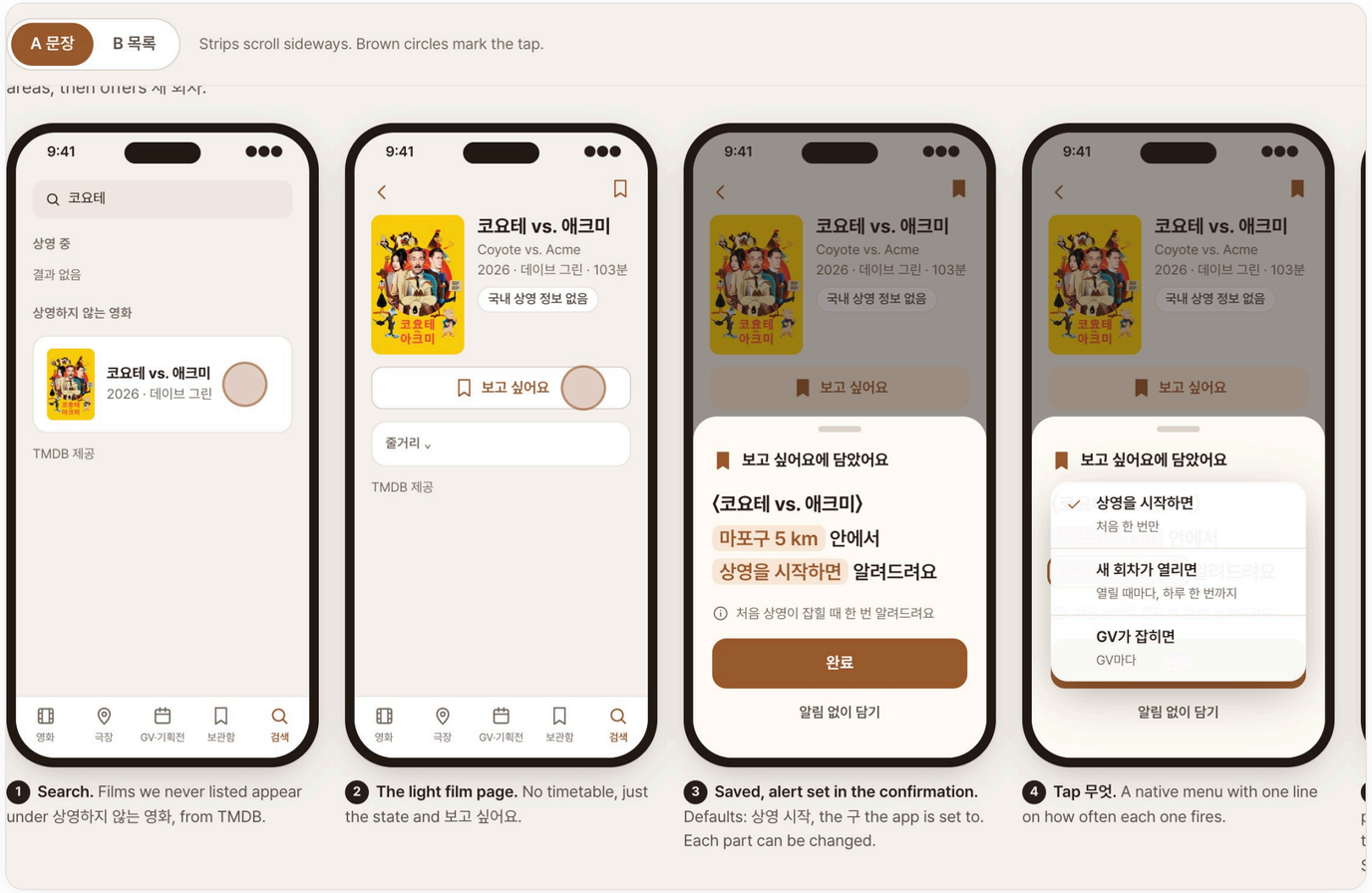
“Let me picture this”

> Add an architecture diagram to the artifact and walk me through the flow, so I can see how it works.



AGENT → YOU

One example, screen by screen



1 **Search.** Films we never listed appear under 상영하지 않는 영화, from TMDB.

2 **The light film page.** No timetable, just the state and 보고 싶어요.

3 **Saved, alert set in the confirmation.** Defaults: 상영 시작, the 구 the app is set to. Each part can be changed.

4 **Tap 무엇.** A native menu with one line on how often each one fires.

One film, from search to a saved alert, in mockups made before the real screens.

AGENT → YOU

Two options, side by side

> Build both, put screenshots side by side, and tell me which you’d pick.

다음 상영 비교

규칙은 그대로예요 (지금 출발해서 5분 여유 있게 닿는 회차만). 바뀌는 건 **언제 나가야 하는지**를 보여주는 방식과, 얼마나 크게 보여줄지예요. 같은 시각, 같은 위치(망원동)의 실제 데이터예요.

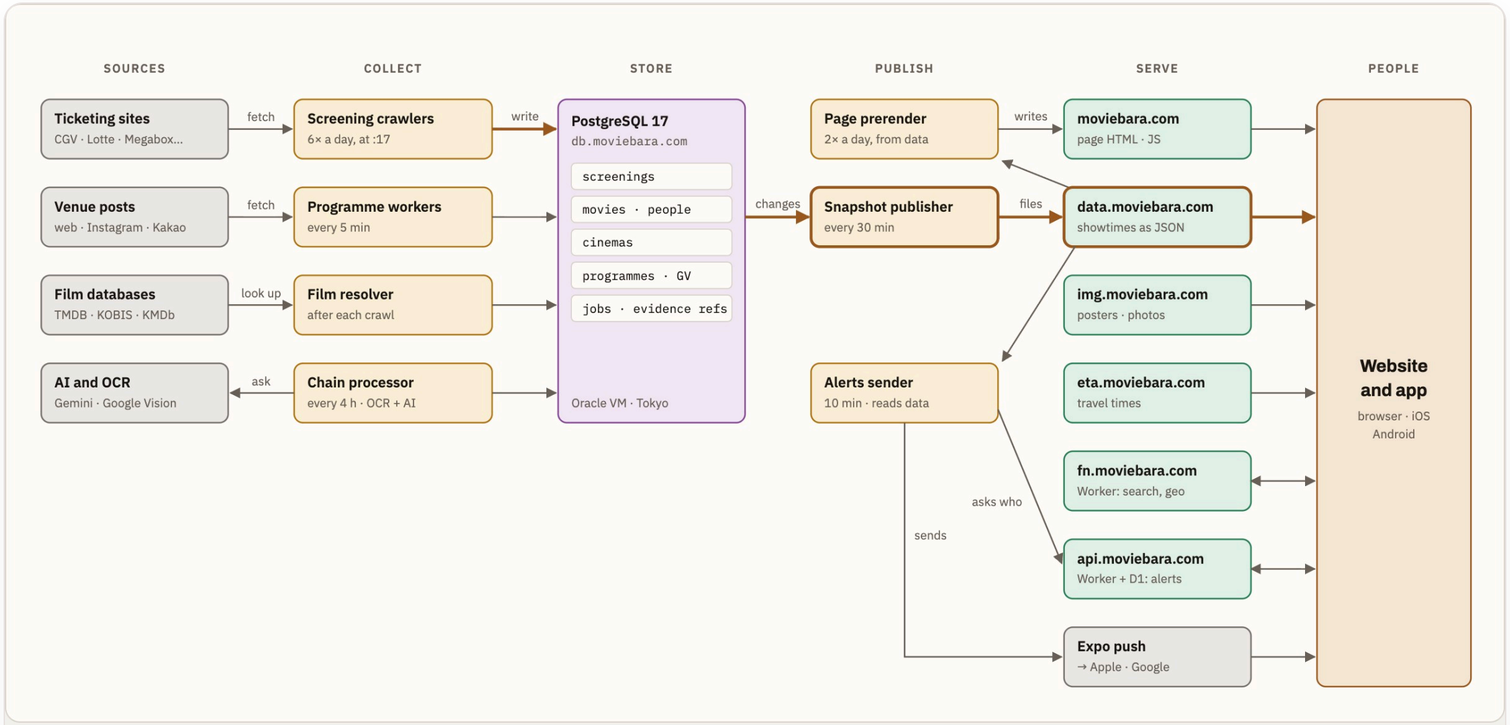
홈의 줄



지금: 이동 25분 | A: 한 줄 그대로, "8분 뒤 출발" | B: 카드 줄 (옆으로 넘김)

- **A** — 예전에 정한 한 줄 설계를 그대로 두고, 결정에 필요한 숫자만 바꿔요. 이동시간은 아이콘으로 남아요. 화면을 더 차지하지 않아요.
- **B** — 두세 개가 한눈에 보여서 고르는 재미가 생겨요. 대신 영화 포스터 줄이 약 100px 내려가요.

One map of the whole system



Sources are read by Lambdas in Seoul, which write to Postgres in Tokyo. The snapshot publisher turns changes into files on `data.moviebara.com`. Two other jobs read those files back: the prerender draws HTML pages for search engines, and the alerts sender decides which pushes to send.

Where each part runs and how data moves between them, updated as the system changes.

Questions I ask

> Walk me through your proposed design, before and after.

> How is this data stored and read? Give me a short overview.

> I’ve lost the context. Explain that rule again with an example.

> Is this a standard design for this kind of problem?

> Where are we overall? What’s done and what’s left?

If you can’t explain what it’s about to build, keep asking.

YOUR FIRST STEP

Prompts for your first slice

You → agent

```
Read docs/01-pitch.md and docs/02-system-design.md.
I want to build this first: [your sentence].
Ask me what's unclear before planning. No code yet.
```

Agent → you

```
Draw the flow as a diagram in an artifact, from the tap to the screen,
and walk me through it with one real example.
List the files, the tables and the tests you'll add.
```

PRODUCTION: 무비바라 CLAUDE.MD

Standing instructions

```
## Fix data where it is produced
When the site shows wrong data, fix the crawler or the
database that produced it. Don't patch it in the page.
```

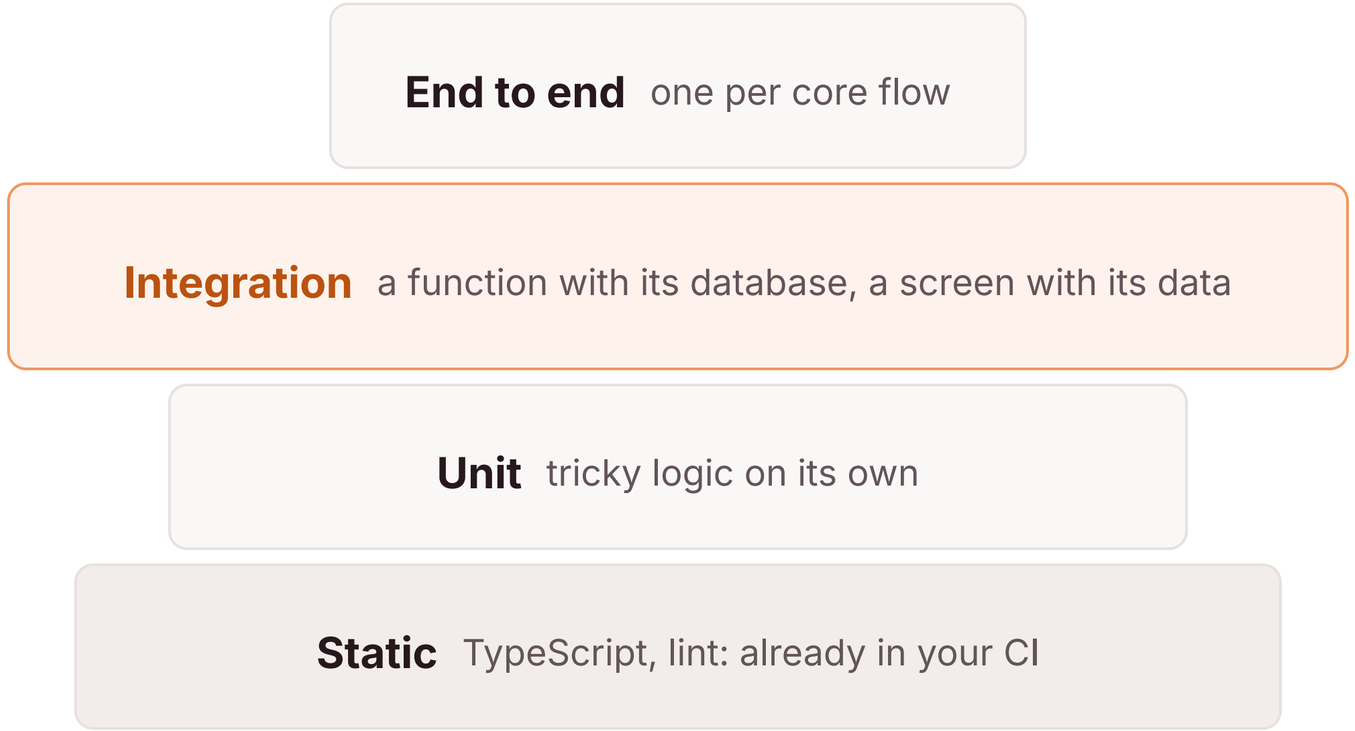
The agent reads this file in every session. Each rule was added after the same mistake happened more than once.

04

What tests should do

The agent writes them. You decide what they check.

Mostly integration



Kent C. Dodds — Write tests. Not too many. Mostly integration.
kentcdodds.com/blog/write-tests

What a good test looks like

One behavior	a test per behavior, not per function
From outside	call it the way the app calls it
Given, when, then	set up, do one thing, check the result
A name that says it	the name is the first thing you read when it fails
No logic inside	no loops or ifs: readable at a glance

A test should only need to change when the expected behavior changes.

Software Engineering at Google, ch.12 Unit Testing
abseil.io/resources/swe-book/html/ch12.html

YOUR FIRST STEP

A backend test, given / when / then

```
test("a saved film shows in the saved list", async () => {
  // given: an empty test backend
  const backend = createTestBackend();
  // when: the user saves a film
  await backend.saveFilm({ filmId: "f1" });
  // then: the list has it
  const saved = await backend.listSaved();
  expect(saved).toMatchObject([{ filmId: "f1" }]);
});
```

Each backend sets up a test backend its own way. Ask the agent to set it up for yours.

YOUR FIRST STEP

Read the names, not the code

```
✓ a saved film shows in the saved list
✓ saving the same film twice keeps one
✓ removing a film takes it off the list
✓ the saved screen says so when there is nothing saved
✓ the saved screen offers a retry when loading fails
```

Ask the agent for the list of test names first. A behavior missing from the list is a missing test.

Let the agent see your app

1. **Add the Expo MCP server**

   ```
   claude mcp add --transport http expo https://mcp.expo.dev/mcp
   ```

2. **Add the local tools**

   ```
   npx expo install expo-mcp --dev
   ```

3. **Start with MCP on**

   ```
   EXPO_UNSTABLE_MCP_SERVER=1 npx expo start
   ```

It can tap, take screenshots and read the logs, so ask it to show you the result.

A bug gets a failing test first



The test reproduces the bug, then keeps it from coming back.

PRODUCTION: 무비바라 APP

Test names that describe behavior

- ✓ the list shows loading, then a retry if it fails
- ✓ swiping removes one entry; “clear all” asks first
- ✓ an empty saved list still links to all cinemas

74

test files, app

176

test files, site

492

test files, crawler

PRODUCTION: 무비바라

Tests run before every push

git push → **Run the tests for what changed** → **Push goes through**

Yours can start smaller: add `npm test` to the CI file you already have.

05

After the first slice

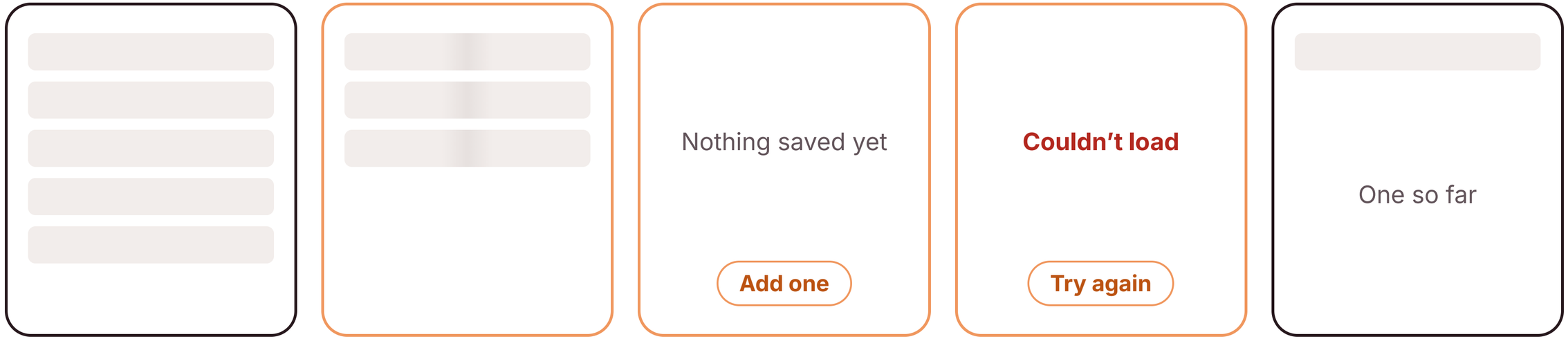
The next slice, and the states in between

Then the next slice, the same way

Pick the next scope → **Sync with the agent** → **Build and check** → **Commit**

Update the scope list as you go. New tasks will show up, and that’s normal.

Every screen that loads data has these states



Ideal
what you designed

Loading
and it must end

Empty
nothing there yet

Error
and a way out

Partial
some data

Turn wifi off and try your flow. Ask the agent to handle each state it finds.

Scott Hurff — the UI Stack (O’Reilly interview)
oreilly.com/ideas/scott-hurff-on-designing-at-tinder

06

Assignment 3

Your first slice, end to end · due 11/6, 23:59

1. The slice

One slice, working

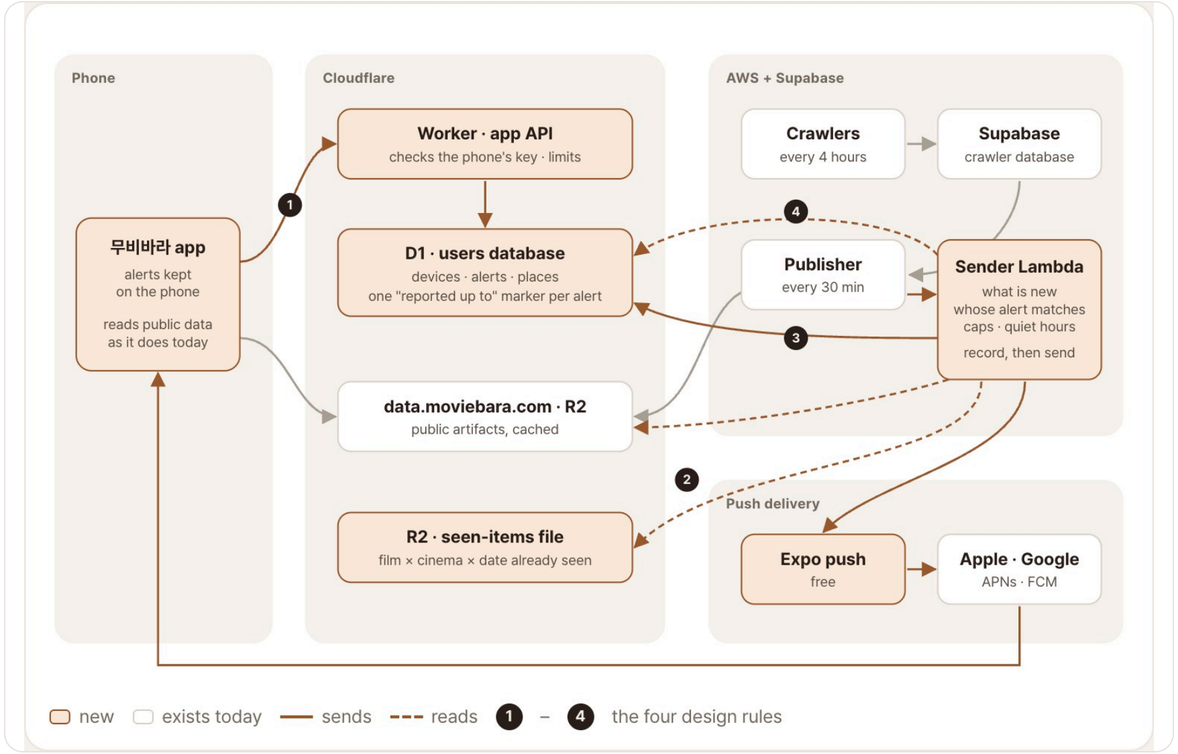
```
When I ___, it is saved to ___,
and it shows on ___.
```

Running on your phone, with real data.

2. The diagram

A diagram of how it works

```mermaid
flowchart LR
  A[Film page] -->|1 tap 보고 싶어요| B(saved.add)
  B -->|2 insert| C[(saved table)]
  C -->|3 saved.list| D[보관함 screen]
```

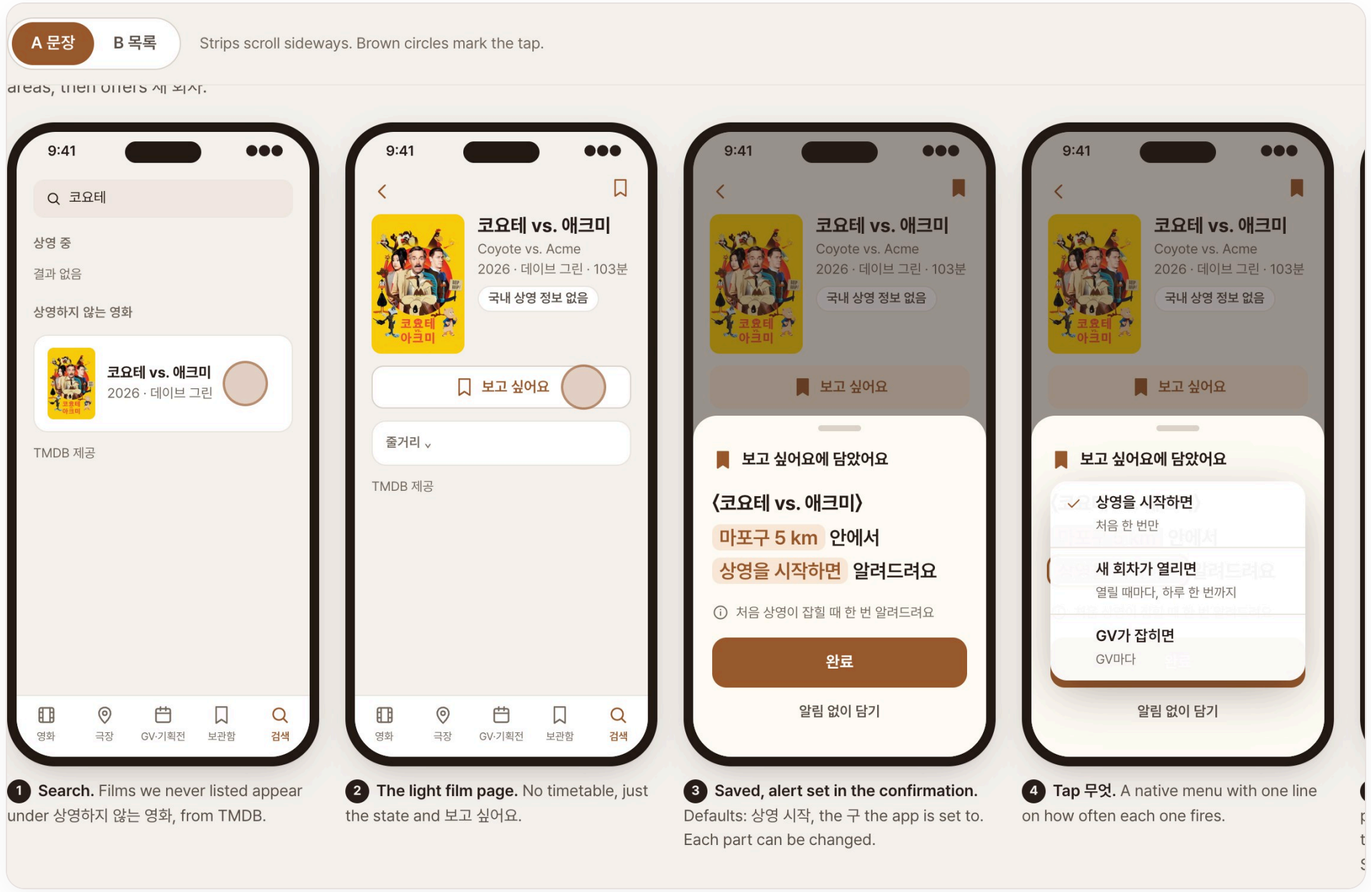


Like this: the parts, and numbered steps for one example

Have the agent draw it, then check it against what you built. A PNG in `docs/`, or Mermaid, which GitHub renders in the PR.

3. The screenshots

The same example, on your phone



1 **Search.** Films we never listed appear under 상영하지 않는 영화, from TMDB.

2 **The light film page.** No timetable, just the state and 보고 싶어요.

3 **Saved, alert set in the confirmation.** Defaults: 상영 시작, the 구 the app is set to. Each part can be changed.

4 **Tap 무엇.** A native menu with one line on how often each one fires.

One screenshot per step, numbered like the diagram, plus one of the error state.

What to hand in

1. **One file** `docs/03-first-slice.md`
   The sentence, the diagram, the numbered screenshots.

2. **The code**, in the same repo
   Runs on your phone. CI green.

Due 11/6, 23:59. New branch, one PR. Feedback comes back as review comments.

Classes and due dates

9/12	01	**Problem**
9/22		**Pitch & sketch** due 23:59
9/29	02	**Design & setup**
10/30		**Design & setup** due 23:59
10/31	03	**Build end to end** today
11/6		**End to end** due 23:59
11/7	04	**MVP**
11/13		**MVP** due 23:59
11/14	05	**Ship**
11/20		**Release** due 23:59
11/21	06	**Demo day**

Reading

Shape Up, Ryan Singer	ch.11 Get One Piece Done ch.12 Map the Scopes
Write tests. Not too many. Mostly integration., Kent C. Dodds	What kind of tests to write
Unit testing, Expo	Setting it up on your stack
AI 시대의 개발자를 꿈꾸는 학생 개발자에게, 정상혁 (NAVER D2)	Careers in the AI era
Vibing a Non-Trivial Ghostty Feature, Mitchell Hashimoto	Plan first, then every session shown

References

technically.dev — On Frontends + Backends
technically.dev/posts/software-eng-vibe-coders-frontends-backends

Ryan Singer — Shape Up, ch.12 Map the Scopes
basecamp.com/shapeup/3.3-chapter-12

Ryan Singer — Shape Up, ch.11 Get One Piece Done
basecamp.com/shapeup/3.2-chapter-11

Henrico Dolfing — Start your project with a walking skeleton
henricodolfing.com/2018/04/start-your-project-with-walking-skeleton.html

Claude Code — Best practices
code.claude.com/docs/en/best-practices

Expo — Using MCP with Expo
docs.expo.dev/mcp

Mitchell Hashimoto — Vibing a Non-Trivial Ghostty Feature
mitchellh.com/writing/non-trivial-vibing

Mitchell Hashimoto — My AI Adoption Journey
mitchellh.com/writing/my-ai-adoption-journey

Testing Library — Guiding Principles
testing-library.com/docs/guiding-principles

Software Engineering at Google — ch.12 Unit Testing
abseil.io/resources/swe-book/html/ch12.html

Kent C. Dodds — Write tests. Not too many. Mostly integration.
kentcdodds.com/blog/write-tests

Expo — Unit testing with Jest
docs.expo.dev/develop/unit-testing

theyawns.com — Shipping a Real App in 18 Days
theyawns.com/2026/06/24/claude-code-for-ios-shipping-a-real-app-in-18-days

O'Reilly — Scott Hurff on designing at Tinder
oreilly.com/ideas/scott-hurff-on-designing-at-tinder

NEXT

04
MVP

11/7 13:00